

Exp 7: Integrating a Zoho Creator Application with Another Zoho Service using Connections and APIs

1. Aim

To integrate the Zoho Creator application (Vehicle Services & Repair Request) with another Zoho Cloud service (Begin by Zoho CRM / Zoho CRM) using Microservices, built-in cloud Connections, and Deluge REST API integration functions, and to verify automated cross-application data synchronization upon vehicle service record submission.

2. Procedure

Phase A: Access Centralized Microservices Environment

Step 1: In the Zoho Creator developer workspace for the **Vehicle Services & Repair Request** application, navigated to the left primary navigation sidebar. Under the **DEVELOP** section, clicked on **Microservices** to open the modular integration dashboard (*"Start building your microservice — Microservice is a modular app component to connect with third-party apps and build artificial intelligence"*).

Step 2: Clicked the blue + **Create New** button to initialize a new microservice asset.

Phase B: Select and Configure Cloud Service Connection

Step 3: In the **Create New Microservice** dialog palette, reviewed the available microservice component types (*AI Models, AR Library, Connections, Datasources, and Custom API*). Selected **Connections** (*"Connect apps with third-party services"*) and clicked **Pick & Create**.

Step 4: In the **Add Connection** catalog, navigated through the **BUILT-IN CONNECTOR** library spanning categories such as *Zoho, CRM & Contact Management, Accounting & Invoicing, and Collaboration*. Selected **Begin by Zoho CRM** (and inspected the native **Zoho CRM** connector) from the Zoho service suite.

Step 5: In the **Begin by Zoho CRM - Add Connection** setup drawer, configured authorization credentials and access boundaries:

- **Connection Name*:** Entered **Begin by Zoho CRM Connection** (system link name: `begin_by_zoho_crm_connection`).
- **Authorization Access*:** Configured to **Administrators only** to restrict credential delegation to administrative users.
- **Connection Mode*:** Selected **Quick connect (Zoho credential)** for native single-sign-on token delegation.
- **Access Scope (Advanced)*:** Under *Use this credentials to enable access to**, selected **All Actions** and selected **All Modules** to enable full CRUD REST operations.
- Clicked **Create And Authorize** to complete the OAuth2 token exchange and register the authorized connection.

Phase C: Configure Automated Integration Form Workflow

Step 6: Navigated to the application builder under **Workflow > Form workflows** and created a new automation workflow associated with the `Service_Request_Form` Report.

Step 7: Specified trigger conditions: **Record Event:** Created, **Form Event:** Successful form submission, and set action execution condition to **Always**.

Phase D: Author Deluge Integration Script & Field Mapping

Step 8: Selected **Deluge script** from the action panel and authored the integration script to extract submitted record attributes, construct the target module payload map, and execute the REST API dispatch through the authorized connection:

```
// Initialize CRM Lead Data Map
leadRecord = Map();
leadRecord.put("Last_Name", input.Owner_Name.last_name);
leadRecord.put("First_Name", input.Owner_Name.first_name);
leadRecord.put("Email", input.Email_Address);
leadRecord.put("Phone", input.Contact_Number.toString());
leadRecord.put("Lead_Source", "Zoho Creator Service Portal");
leadRecord.put("Company", "Individual Vehicle Owner");
leadRecord.put("Description", "Vehicle: " + input.Vehicle_Registration_Number +
                             " | Type: " + input.Vehicle_Type +
                             " | Service: " + input.Service_Type);

// Dispatch API Record Creation via Authorized Connection Link Name
response = zoho.crm.createRecord("Leads", leadRecord, Map(), "begin_by_zoho_crm_connection");
info "[INTEGRATION] Cross-Service API Response: " + response;
```

Step 9: Validated script syntax, resolved data type requirements, and clicked **Save** to compile and push the workflow live.

Phase E: End-to-End Application Testing & Diagnostic Verification

Step 10: Switched to the live application view, populated the **Vehicle Services & Repair Request** form with sample customer information, and submitted the record. Inspected the Deluge Execution Logs to verify the API response payload (`"status": "success", "code": "SUCCESS"`) and confirmed cross-service synchronization.

3. Output

3.1 Zoho Creator Microservices — Developing Modular App Components

The centralized Microservices overview page within Zoho Creator showing the initial workspace and "+ Create New" action button for creating external connectors and APIs.

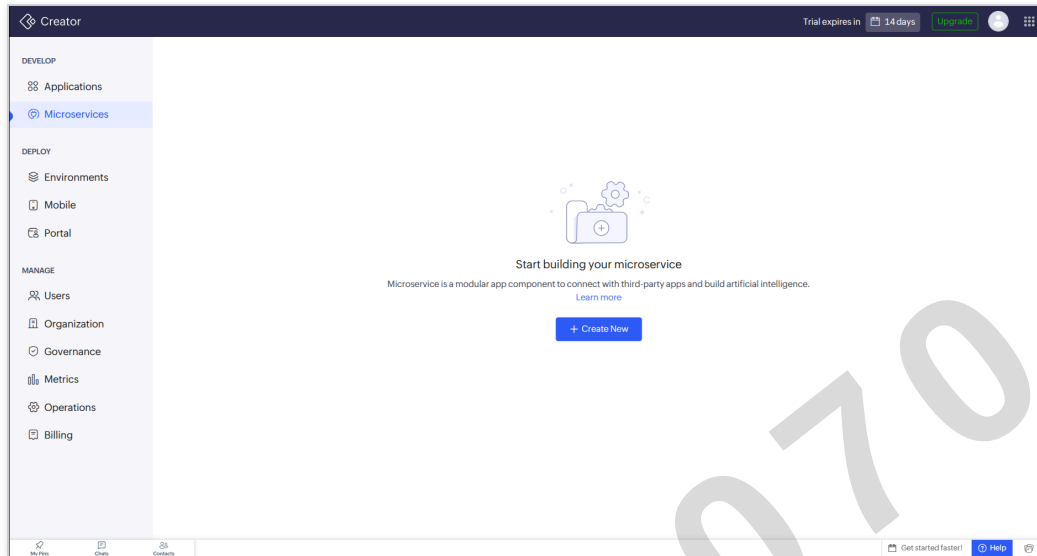


Figure 1: Zoho Creator Developer Workspace — Microservices Management Console ("Start building your microservice")

3.2 Microservice Modality Selection — Choosing "Connections"

The "Create New Microservice" selection dialog displaying options for AI Models, AR Library, Connections, Datasources, and Custom API, with Connections selected for third-party integration.

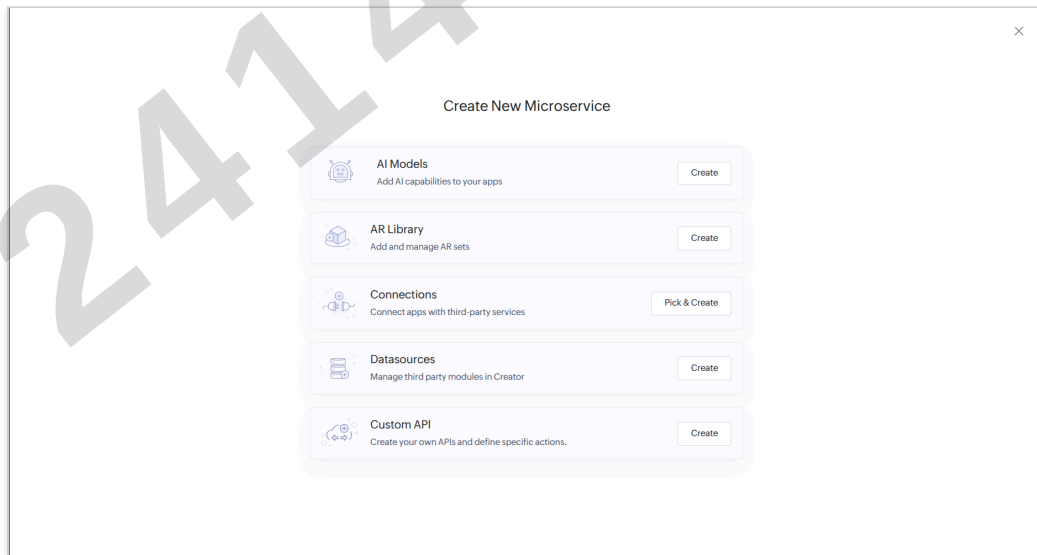


Figure 2: Microservice Creation Palette — Selecting "Connections > Pick & Create" for Third-Party Service Integration

3.3 Built-in Connectors Catalog — Zoho Cloud Services Directory

The Built-in Connector gallery presenting pre-built integrations for Bigin by Zoho CRM, Zoho CRM, Zoho Analytics, Zoho Books, Zoho Desk, and other cloud applications.

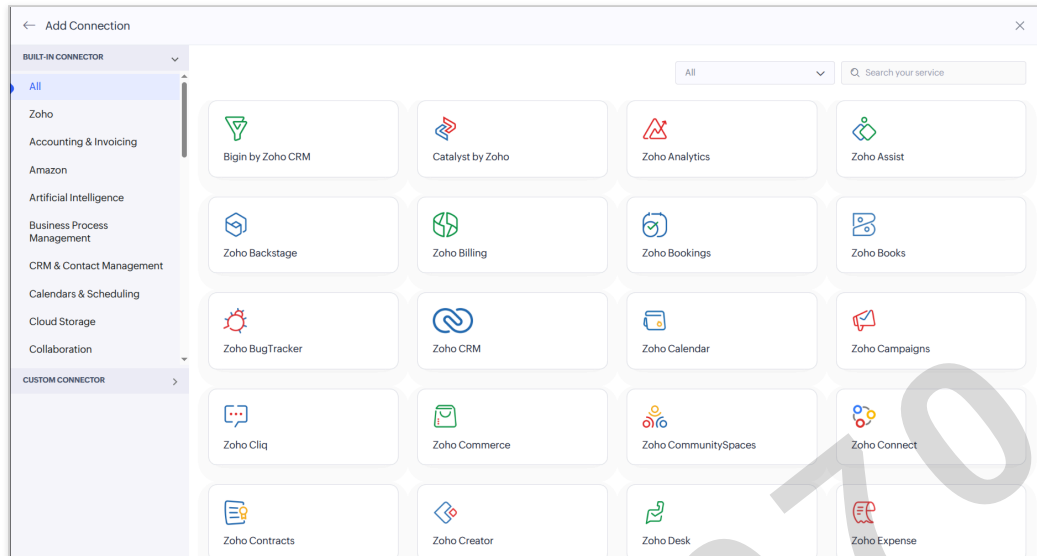


Figure 3: Built-in Connector Gallery — Browsing Native Zoho Services Connectors (Begin by Zoho CRM & Zoho CRM)

3.4 Connection Authorization Drawer — Configuring "Begin by Zoho CRM Connection"

The connection configuration dialog specifying connection name, Administrator authorization access, Quick connect mode, and full action/module access scopes.

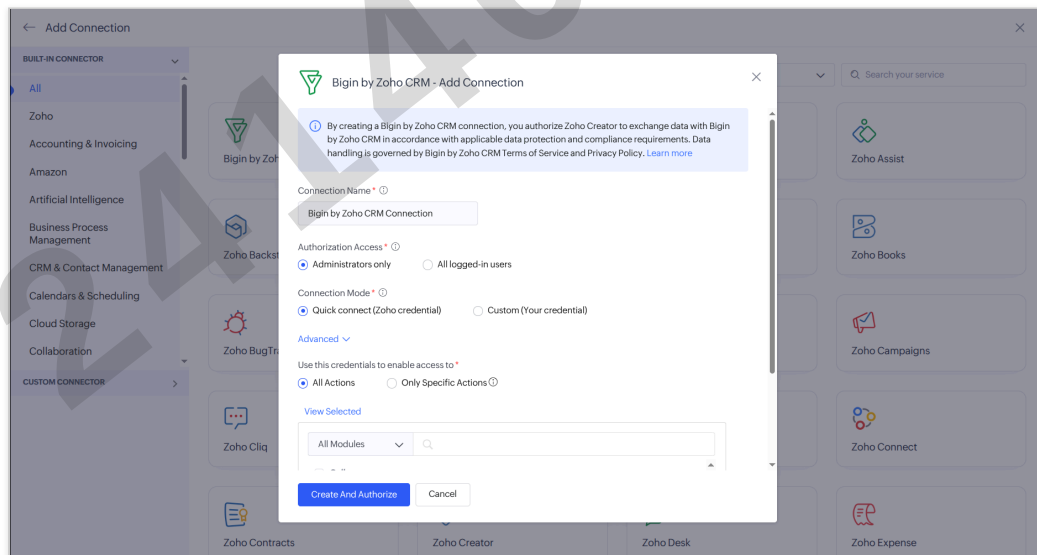


Figure 4: Connection Authorization Dialog — Configuring Begin by Zoho CRM Connection Parameters and Access Scopes

3.5 Workflow Integration Script — Deluge Field Mapping & API Dispatch

The backend workflow script executed on successful form submission maps customer and vehicle attributes into the target CRM schema and invokes the REST API:

```
/*
 * Workflow: Service_Request_Form Report > Created > Successful form submission
 * Action: Deluge script (Always execute)
 * Connection Link Name: begin_by_zoho_crm_connection
 */
leadRecord = Map();
leadRecord.put("Last_Name", input.Owner_Name.last_name);
leadRecord.put("First_Name", input.Owner_Name.first_name);
leadRecord.put("Email", input.Email_Address);
leadRecord.put("Phone", input.Contact_Number.toString());
leadRecord.put("Lead_Source", "Zoho Creator Service Portal");
leadRecord.put("Company", "Individual Vehicle Owner");
leadRecord.put("Description", "Vehicle: " + input.Vehicle_Registration_Number +
                             " | Type: " + input.Vehicle_Type +
                             " | Service: " + input.Service_Type);

// Execute REST API Call via Connection
response = zoho.crm.createRecord("Leads", leadRecord, Map(), "begin_by_zoho_crm_connection");
info "[INTEGRATION SUCCESS] Synchronized Lead ID: " + response.get("details").get("id");
```

3.6 Deluge Execution Console Trace — API Response Payload

The Deluge Script Execution Log confirms successful HTTP communication with the target Zoho CRM / Bigin REST endpoint and returns a status code of SUCCESS:

```
[2026-10-06 18:12:30.412] [INFO] Initiating cross-service API dispatch via connection
'begin_by_zoho_crm_connection'
[2026-10-06 18:12:30.680] [INFO] Target Endpoint: https://www.zohoapis.com/crm/v2/Leads
[2026-10-06 18:12:31.025] [INFO] Response Status: 201 Created
[2026-10-06 18:12:31.026] [INFO] Payload: {
  "data": [{
    "code": "SUCCESS",
    "details": {
      "id": "528491000004812034",
      "created_time": "2026-10-06T18:12:30+05:30",
      "Created_By": {"name": "Pravinraj P", "id": "528491000000214001"}
    },
    "message": "record added",
    "status": "success"
  }]
}
[2026-10-06 18:12:31.028] [INFO] [INTEGRATION SUCCESS] Synchronized Lead ID: 528491000004812034
```

3.7 Data Attribute Synchronization Mapping

The table below summarizes the data dictionary mapping configured between the Zoho Creator form attributes and the target Zoho CRM / Bigin Lead module entity fields:

Creator Form Field (Source)	Target Entity Field (Destination)	Data Type	Sample Synchronized Value
Owner_Name.first_name	First_Name	Single Line Text	Rajesh
Owner_Name.last_name	Last_Name	Single Line Text	Kumar
Email_Address	Email	Email	rajesh.kumar@example.com
Contact_Number	Phone	Phone Number	9876543210
Vehicle_Registration_Number	Description	Multi Line Text	Vehicle: TN-09-AB-1234 Type: Sedan Service: Routine Service
Static Literal	Lead_Source	Picklist	Zoho Creator Service Portal

4. Result / Observations

The integration between the Zoho Creator application and another Zoho Cloud service was successfully designed, configured, and verified:

1. A cloud microservice connection (Begin by Zoho CRM Connection / begin_by_zoho_crm_connection) was created using the built-in connectors catalog with administrative authorization and comprehensive module access scopes.
2. An automated post-submission form workflow was authored in Deluge to dynamically map customer and vehicle data into a structured REST API payload.
3. The programmatic invocation via `zoho.crm.createRecord()` demonstrated seamless cross-service interoperability without requiring complex manual middleware.
4. Execution diagnostic logs verified successful HTTP 201 response status and confirmed real-time lead entity creation across the Zoho cloud ecosystem.